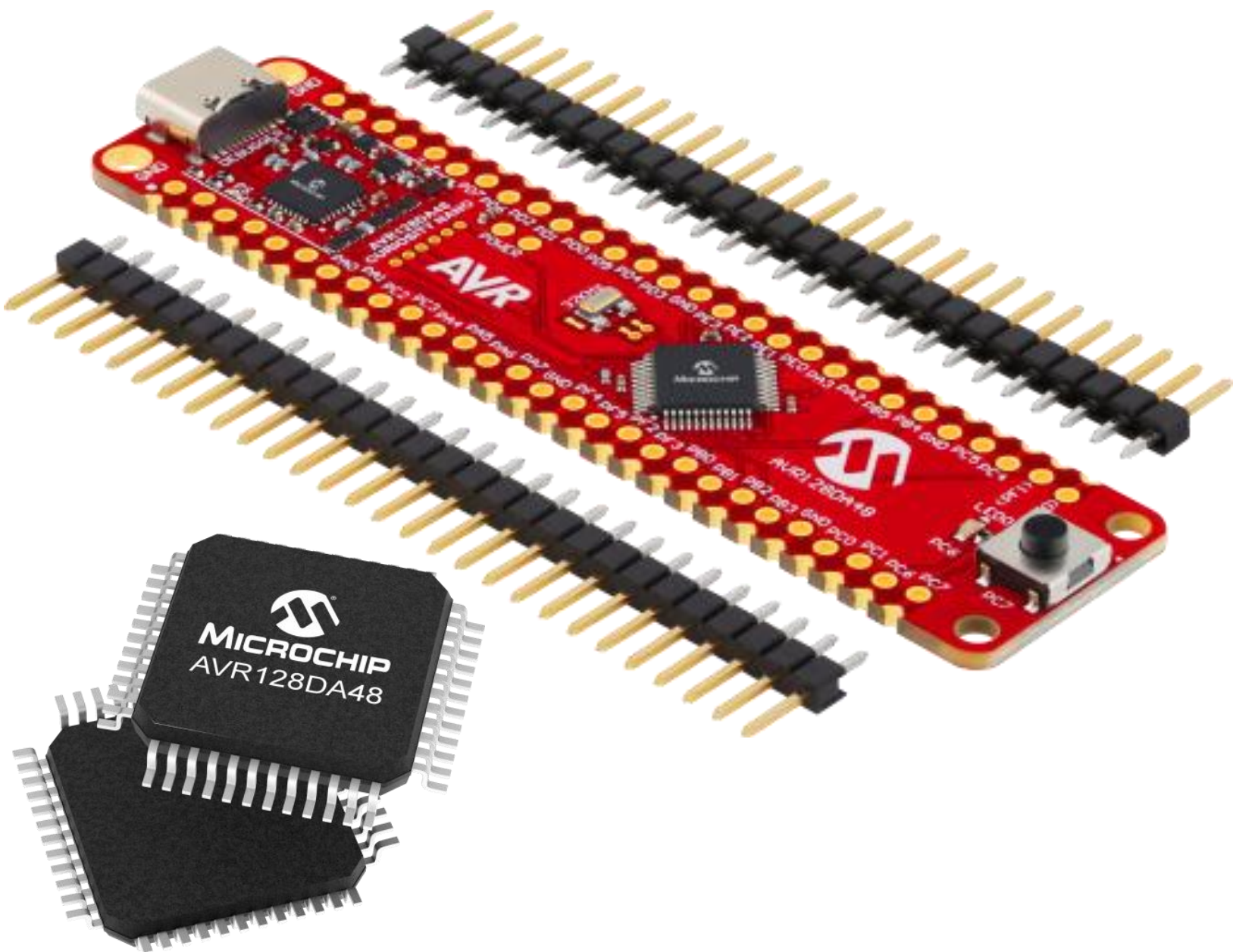


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 1 - Course Introduction and Board Bring-Up



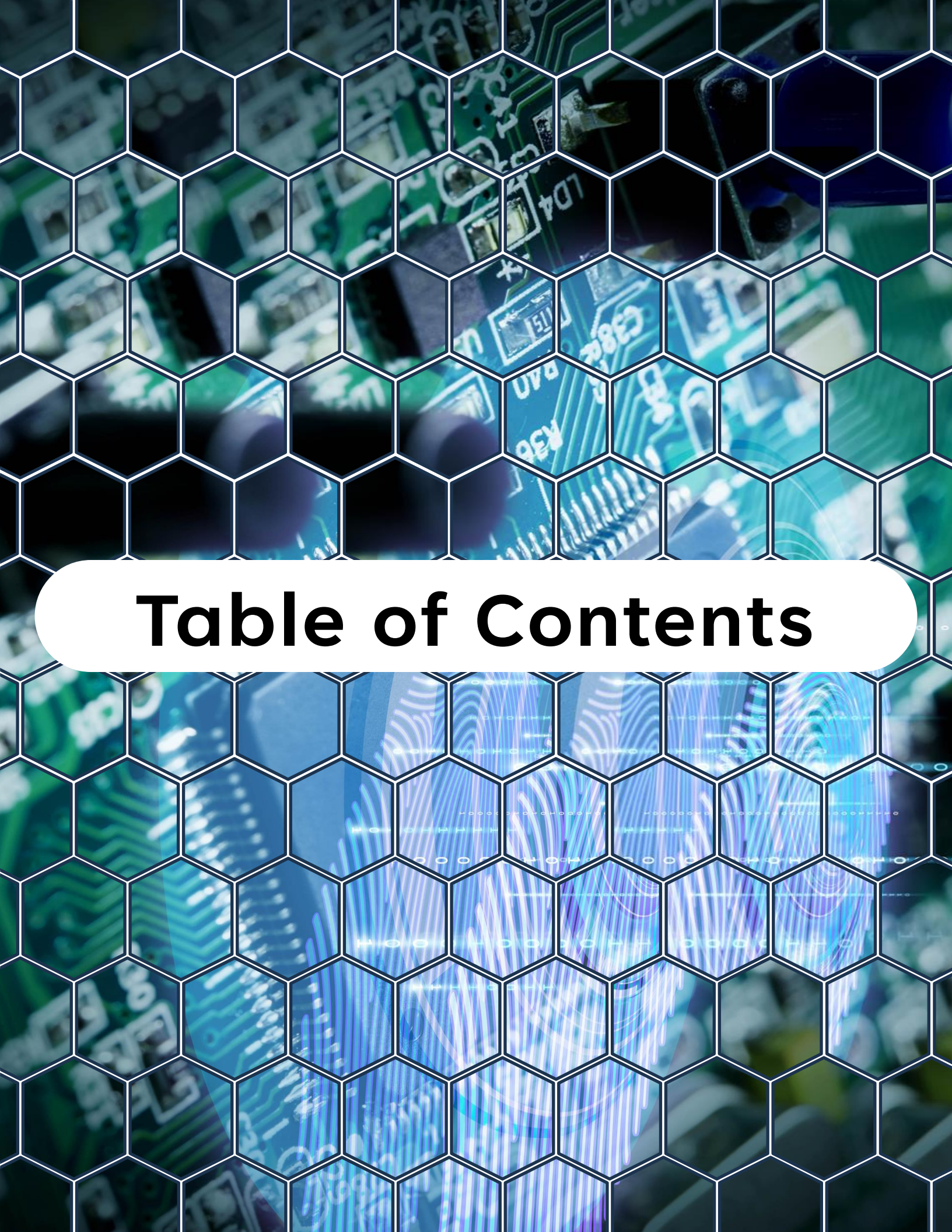
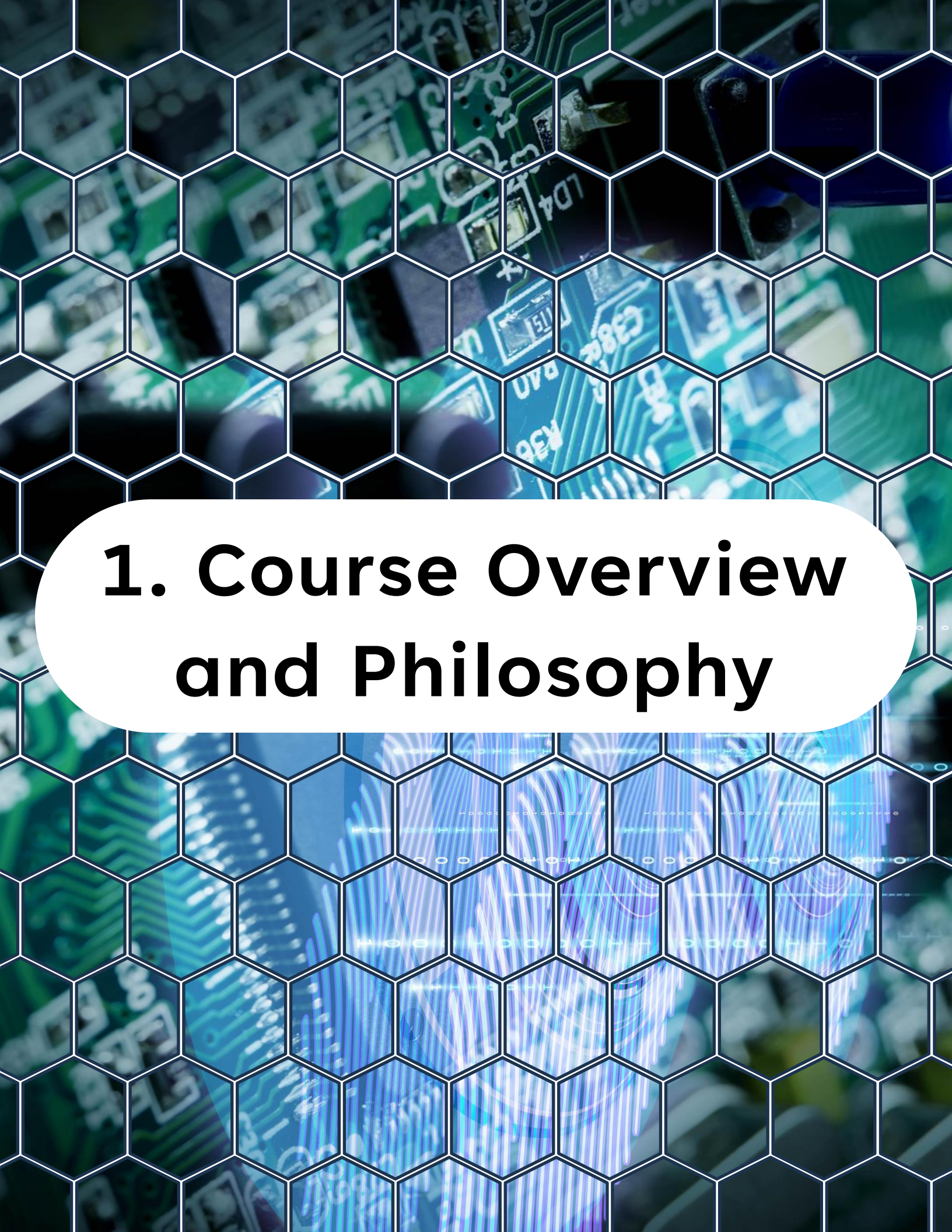


Table of Contents

Table of Contents

1. Course Overview and Philosophy
2. Why AVR128DA48 and Curiosity Nano
3. Development Environment Setup
4. Understanding the Board Hardware
5. Creating a Clean Bare-Metal Project
6. Building and Flashing the Firmware
7. Verifying Successful Bring-Up
8. What You Learned Today
9. What Comes Next



1. Course Overview and Philosophy

1. Course Overview and Philosophy

Welcome to **Bare-Metal Embedded Systems Development with AVR128DA48**.

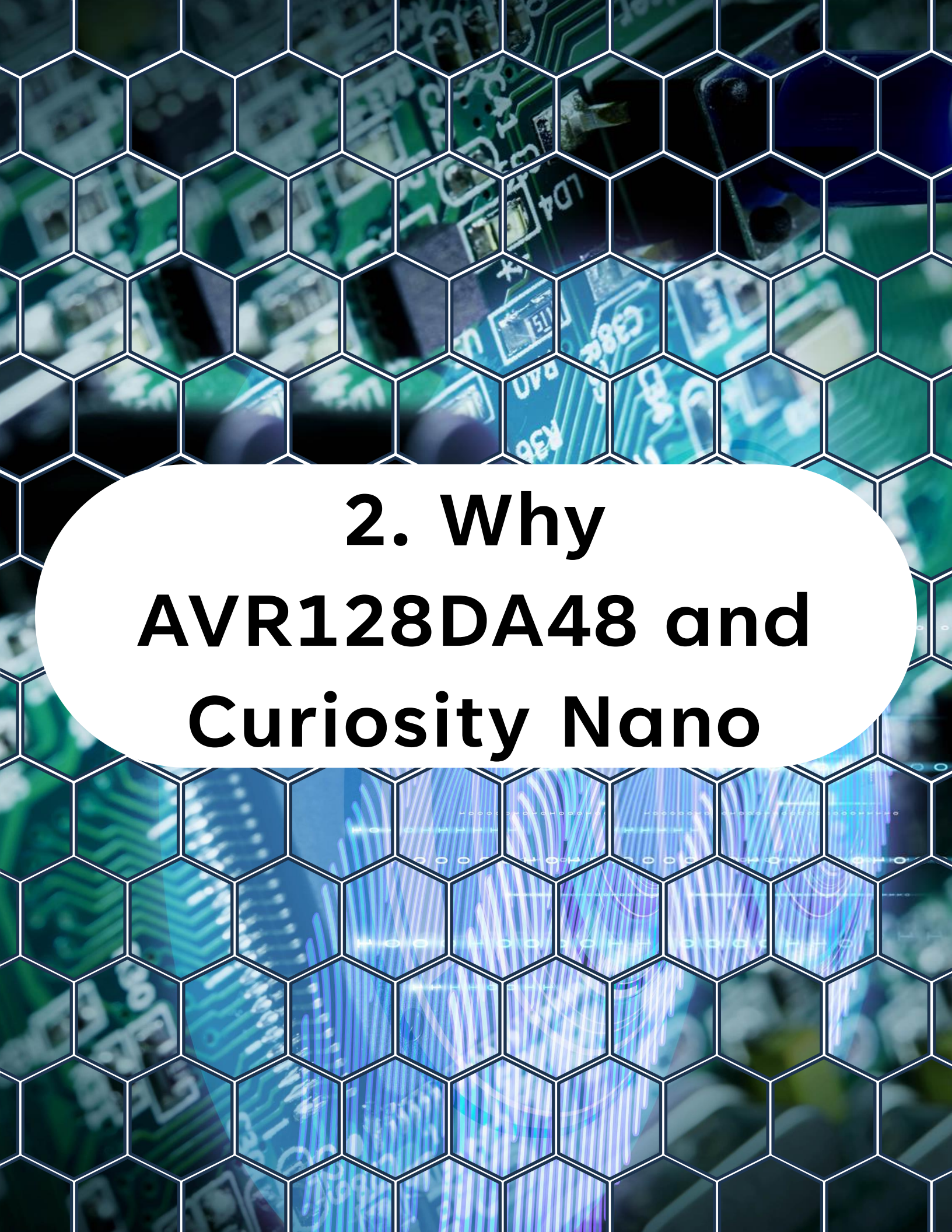
This course is intentionally designed to stay close to the hardware. You will not use Arduino, ASF, MCC code generation, or high-level frameworks. Every peripheral will be configured directly through registers, exactly as defined in the datasheet.

The goal is simple: by the end of this course, you should be able to open the datasheet of any modern AVR device, understand its peripherals, and write clean, deterministic firmware with confidence.

1. Course Overview and Philosophy

Why bare metal?

- You gain full control over timing and resources
- You understand what the hardware is actually doing
- You can debug problems without guessing
- Your code becomes portable and predictable
- This mindset is critical for professional embedded systems development.



2. Why AVR128DA48 and Curiosity Nano

2. Why AVR128DA48 and Curiosity Nano

The **AVR128DA48** is part of Microchip's modern AVR DA family. It represents a major evolution over classic ATmega devices while keeping the simplicity that made AVR popular.

Key reasons for choosing this MCU:

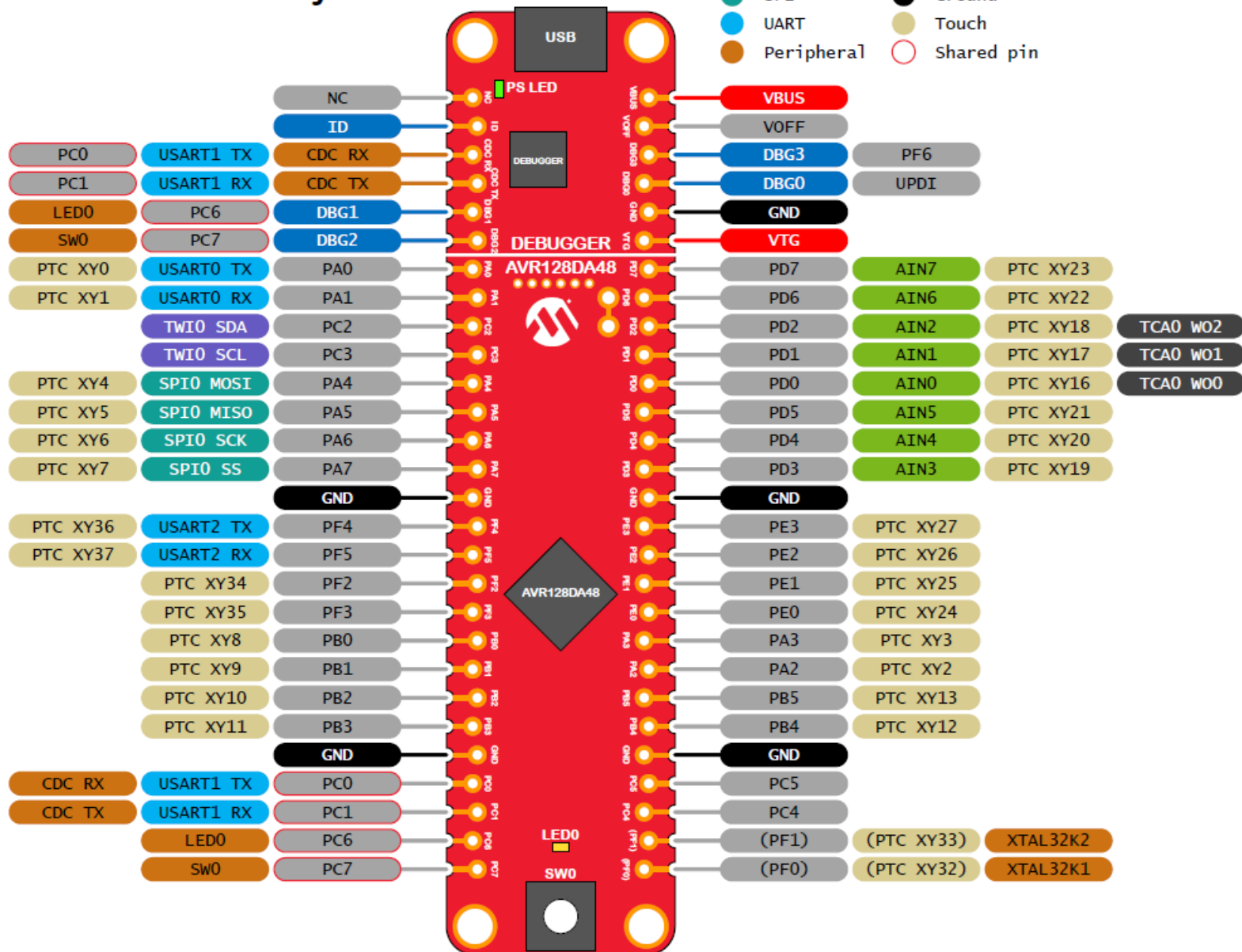
- Rich peripheral set (ADC, DAC, TCA, TCB, RTC, AC, EVSYS)
- Up to 128 KB Flash and generous SRAM
- Clean and consistent register architecture
- Excellent datasheet and reference manual
- Low cost and long-term availability

2. Why AVR128DA48 and Curiosity Nano

The AVR128DA48 Curiosity Nano board complements the MCU perfectly:

- On-board UPDI debugger and programmer
- USB powered, no external tools required
- Many MCU pins exposed
- Clean reference design

This makes it ideal for learning and for building real prototypes.





3. Development Environment Setup

3. Development Environment Setup

Required Software

You will use one of the following IDEs:

- Atmel Studio 7
- Microchip Studio

Both environments support the AVR DA family and integrate seamlessly with the Curiosity Nano debugger.

What We Will NOT Use

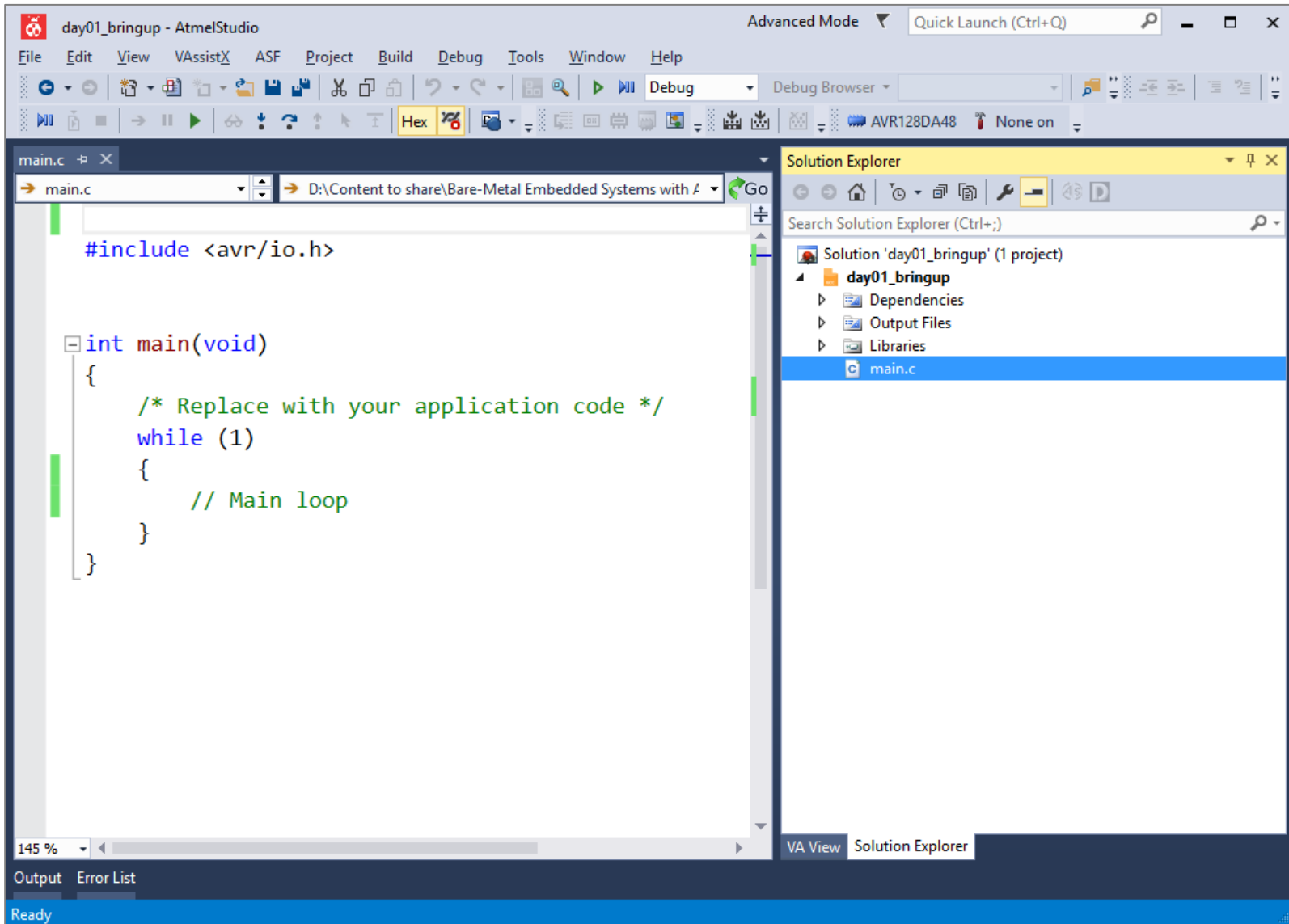
This course does NOT use:

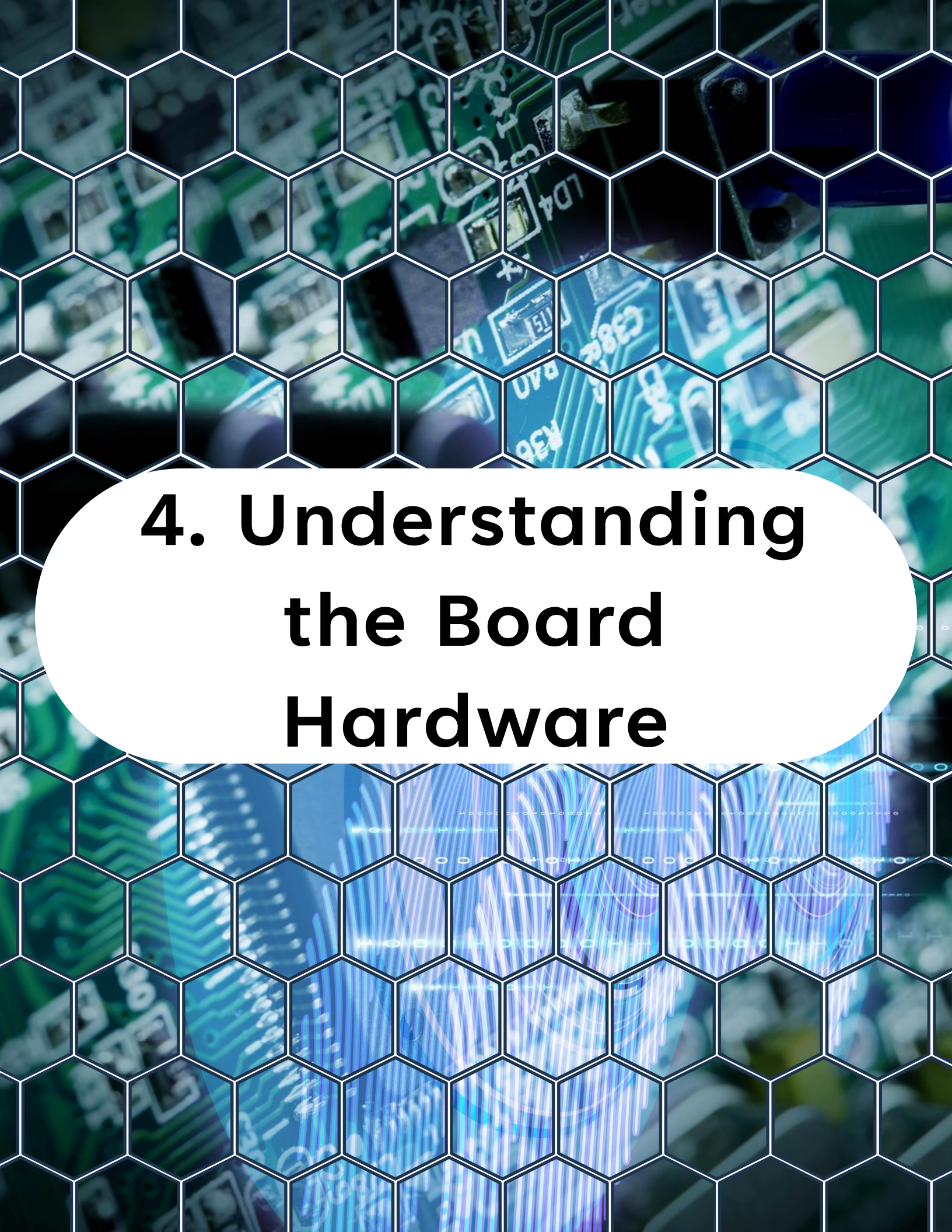
- Arduino core
- MCC auto-generated drivers

We want full visibility and control over every register write.

3. Development Environment Setup

Atmel Studio 7



The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C38', 'U7A', and 'K3B'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the section title in bold black text.

4. Understanding the Board Hardware

4. Understanding the Board Hardware

Before writing code, it is important to understand what is physically on the board.

Key components on the Curiosity Nano:

- AVR128DA48 MCU
- On-board debugger (UPDI over USB)
- Status LED (connected to a GPIO pin)
- USB connector for power and programming

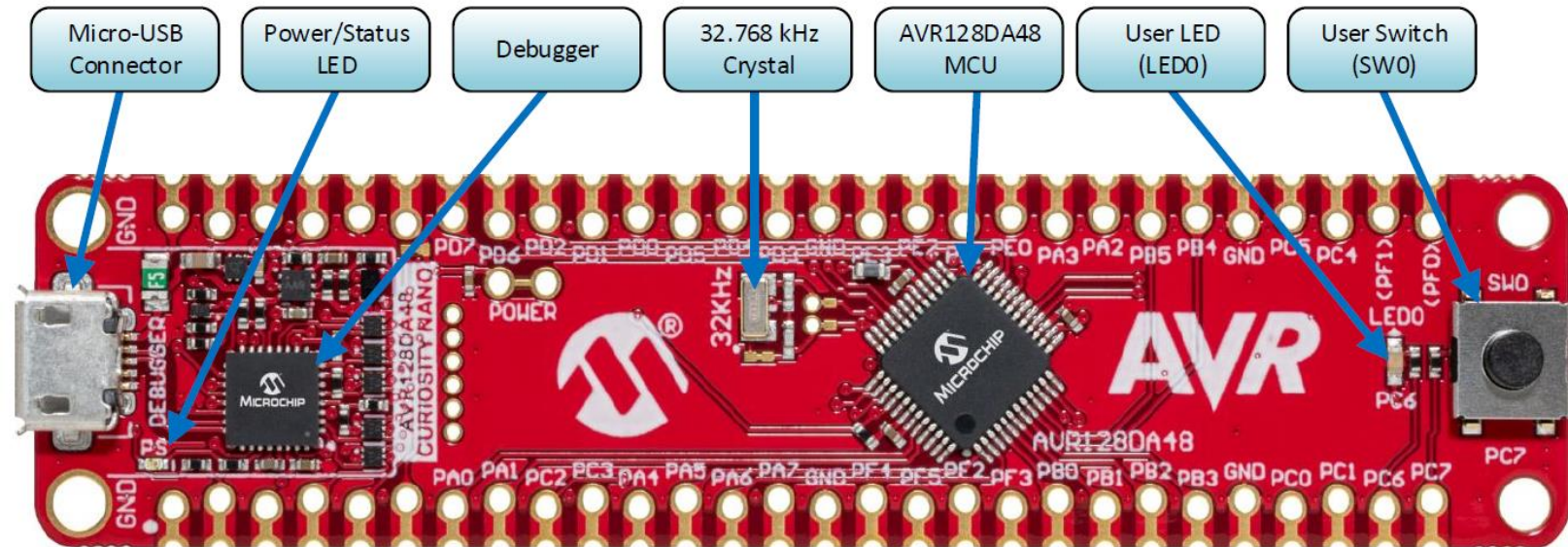
Spend a few minutes with the schematic and pinout. Identify:

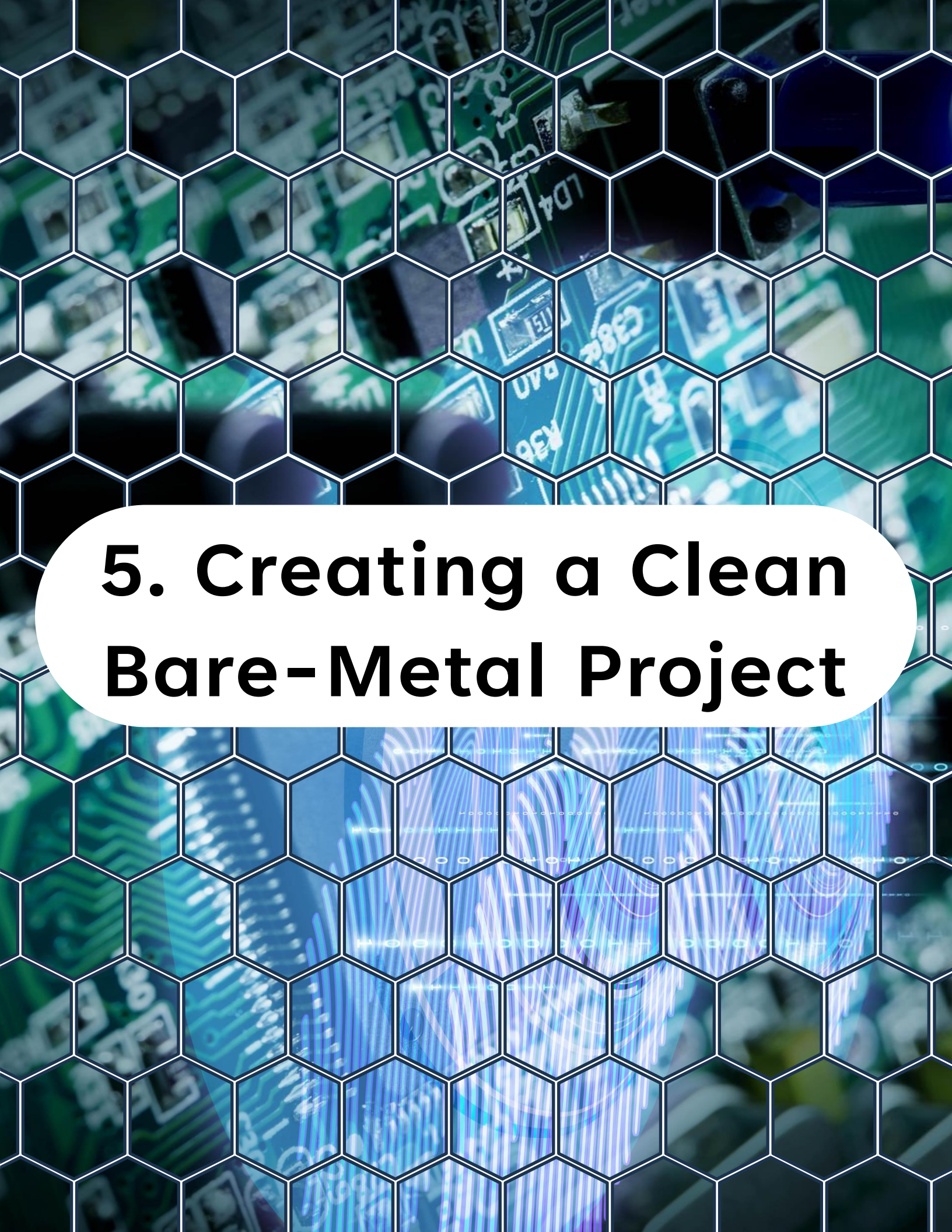
- The LED pin
- Power pins (VDD, GND)
- UPDI pin (used internally by the debugger)

This habit of studying hardware first will save hours of debugging later.

4. Understanding the Board Hardware

AVR128DA48 Curiosity Nano Board Overview





5. Creating a Clean Bare-Metal Project

5. Creating a Clean Bare-Metal Project

Step 1: Create a New Project

1. Open Atmel Studio or Microchip Studio
2. Create a new project
3. Select "GCC C Executable Project"
4. Choose the device: **AVR128DA48**
5. Name the project clearly (example:
day01_bringup)

The IDE will generate a minimal project structure.

5. Creating a Clean Bare-Metal Project

Step 2: Minimal main.c

Replace the contents of main.c with the following minimal skeleton.

```
#include <avr/io.h>

int main(void)
{
    /* Replace with your application code */
    while (1)
    {
        // Main loop
    }
}
```

This is intentional. No delays, no drivers, no interrupts. Just a clean entry point.



6. Building and Flashing the Firmware

6. Building and Flashing the Firmware

1. Connect the Curiosity Nano board via USB
2. Build the project
3. Click "Start Without Debugging" or "Program Device"

If everything is correct:

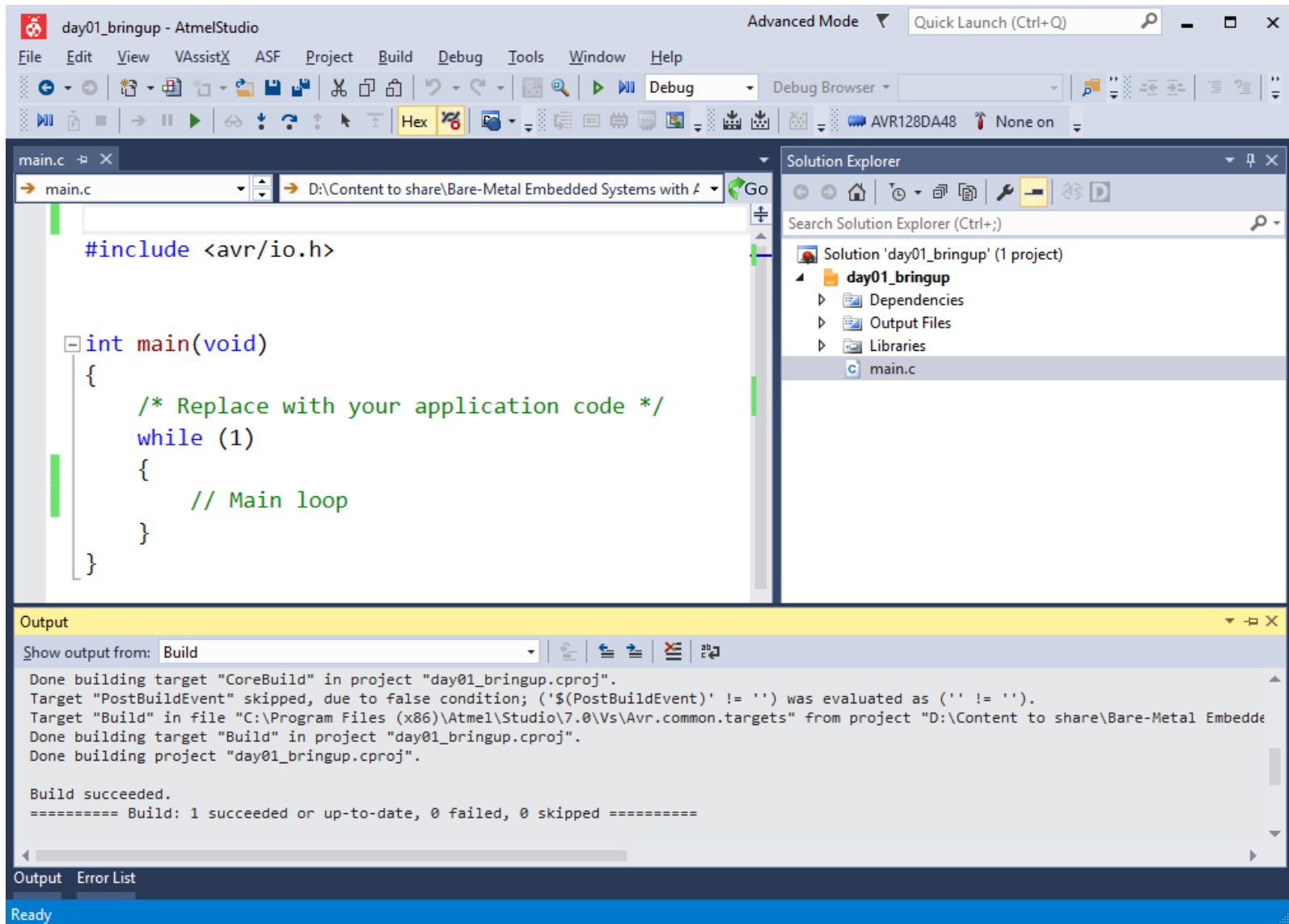
- The build succeeds
- The firmware is flashed
- The MCU runs an empty loop

At this point, the MCU is alive and executing your code.

This is your first milestone.

6. Building and Flashing the Firmware

Build succeeded





7. Verifying Successful Bring- Up

7. Verifying Successful Bring-Up

How do you know the code is running?

For now:

- The debugger connects successfully
- No programming errors occur

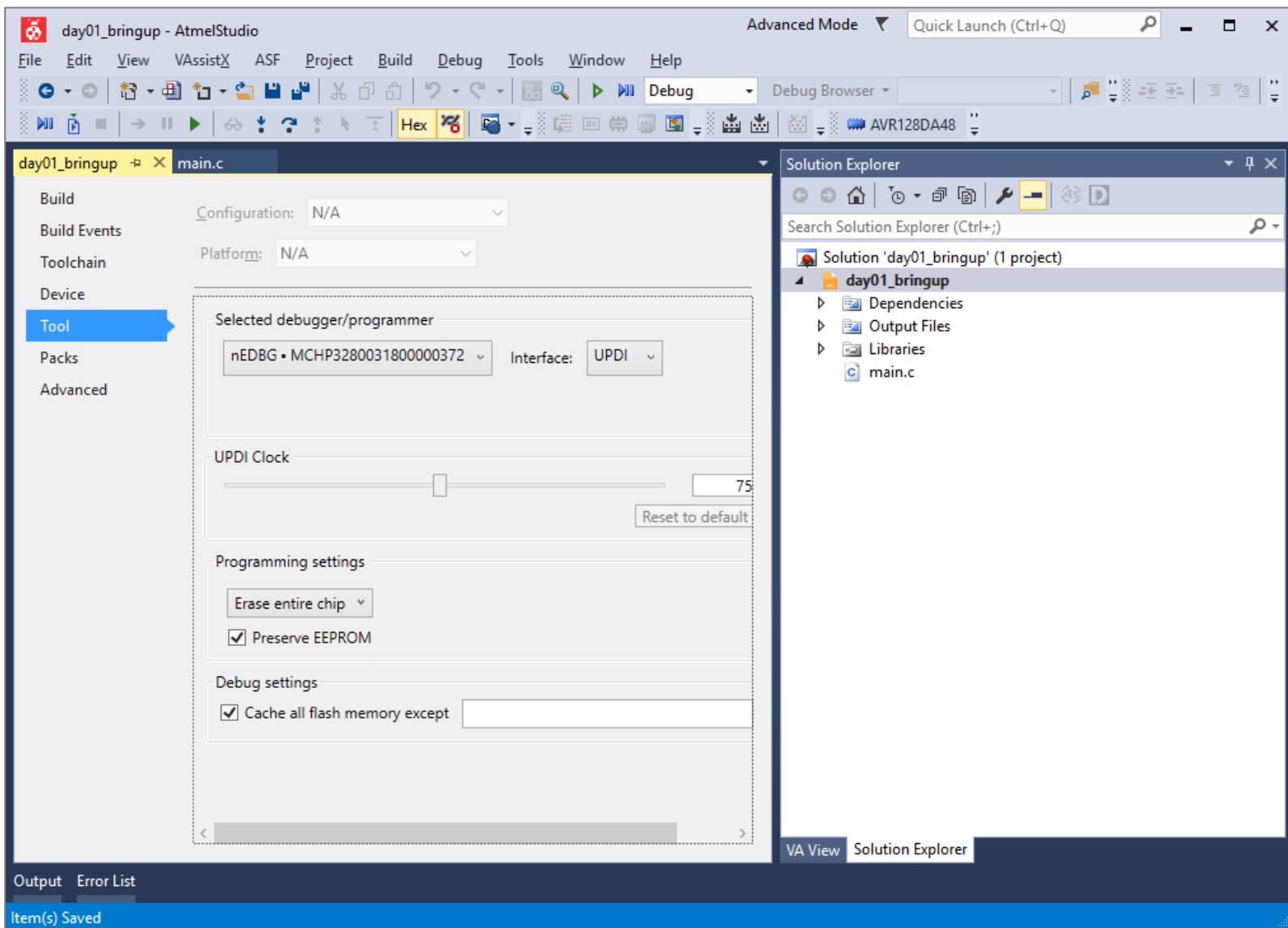
In Day 2 and Day 3, you will verify execution using GPIO toggling and timing measurements.

For professional embedded work, successful bring-up is not flashy. It is quiet, boring, and correct.

That is exactly what you want.

7. Verifying Successful Bring-Up

Selected Debugger/Programmer





8. What You Learned Today

8. What You Learned Today

By the end of Day 1, you have:

- Understood the goals and structure of the course
- Learned why AVR128DA48 is a strong learning platform
- Set up a professional bare-metal development environment
- Created and flashed a clean firmware project

This foundation matters. Every peripheral we touch later will build on this exact structure.



9. What Comes Next

9. What Comes Next

Day 2 - AVR128DA48 Architecture Deep Dive

Next, we will explore:

- CPU core behavior
- Memory map and addressing
- How peripherals are memory-mapped
- How to read the datasheet like an engineer, not a beginner

From this point forward, the datasheet becomes your primary tool.

When you are ready, say the word and we will continue with Day 2.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>